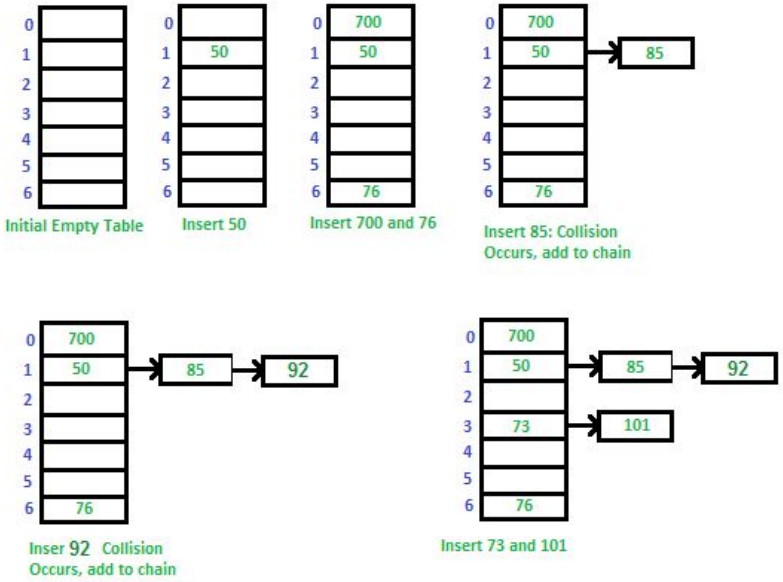
TABELA HASH

Análise de Algoritmos

Função de custo e complexidade

Encadeamento Separado

- **Data Structures:** Uso de `struct No` e `struct TabelaHash` para baldes (buckets) dinâmicos.
- **Função Hash:** Método da divisão `abs(chave % TAMANHO)` para garantir endereços válidos.
- **Inserção O(1):** Adição no início da lista ligada do bucket correspondente.
- **Gestão de Memória:** Funções de liberação para evitar *memory leaks*.



Complexidade

Operação	Melhor caso	Caso Médio	Pior Caso
Busca / Remoção	O(1)	O(1+a)	O(n)
Inserção	O(1)	O(1)	O(n)*

Código em C do Algoritmo

Estrutura e Função Hash

```c
#define TAMANHO 10

typedef struct No {
    int chave;
    struct No* proximo;
} No;

typedef struct {
    No* buckets[TAMANHO];
} TabelaHash;
```

Funções Implementadas

- **funcaoHash()**: calcula índice - O(1)
- **inserir()**: insere, rejeita duplicata
- **buscar()**: retorna nó ou NULL
- **remover()**: remove e libera memória
- **exibir()**: mostra estado da tabela

Lógica de Inserção e Busca

```c
void inserir(TabelaHash* t, int chave) {
    int idx  = funcaoHash(chave);
    No* atual = t->buckets[idx];

    while (atual) {
        if (atual->chave == chave) {
            printf("Chave %d ja existe na tabela.\n", chave);
            return;
        }
        atual = atual->proximo;
    }

    No* novo = (No*) malloc(sizeof(No));
    novo->chave   = chave;
    novo->proximo = t->buckets[idx];
    t->buckets[idx] = novo;
    printf("Inserido: %d --> bucket[%d]\n", chave, idx);
}

No* buscar(TabelaHash* t, int chave) {
    int idx  = funcaoHash(chave);
    No* atual = t->buckets[idx];

    while (atual) {
        if (atual->chave == chave) return atual;
        atual = atual->proximo;
    }
    return NULL;
}
```

O experimento

O experimento consistiu em testar o desempenho do algoritmo com 3 métodos de entradas, crescente, decrescente e aleatório, preenchendo a tabela hash com 10^3,10^4 ,10^5,10^6,10^7 entradas.

Para cada método de entrada foi repetido o teste 13 vezes e da média do resultado foi montado o gráfico.

tipo	n_elementos	execucao	tempo_s	comparacoes	memoria_kb
1	1000	1	0.000040363	0	2208
1	1000	2	0.000025608	0	2208
1	1000	3	0.000025044	0	2208
1	1000	4	0.00004428	0	2208
1	1000	5	0.000024957	0	2208
1	1000	6	0.000025421	0	2208
1	1000	7	0.000025139	0	2208
1	1000	8	0.000025712	0	2208
1	1000	9	0.000025696	0	2208
1	1000	10	0.000023526	0	2208
1	1000	11	0.000025534	0	2208
1	1000	12	0.000026083	0	2208
1	1000	13	0.000031892	0	2208
1	1000	MEDIA	0.000028404	0	2208

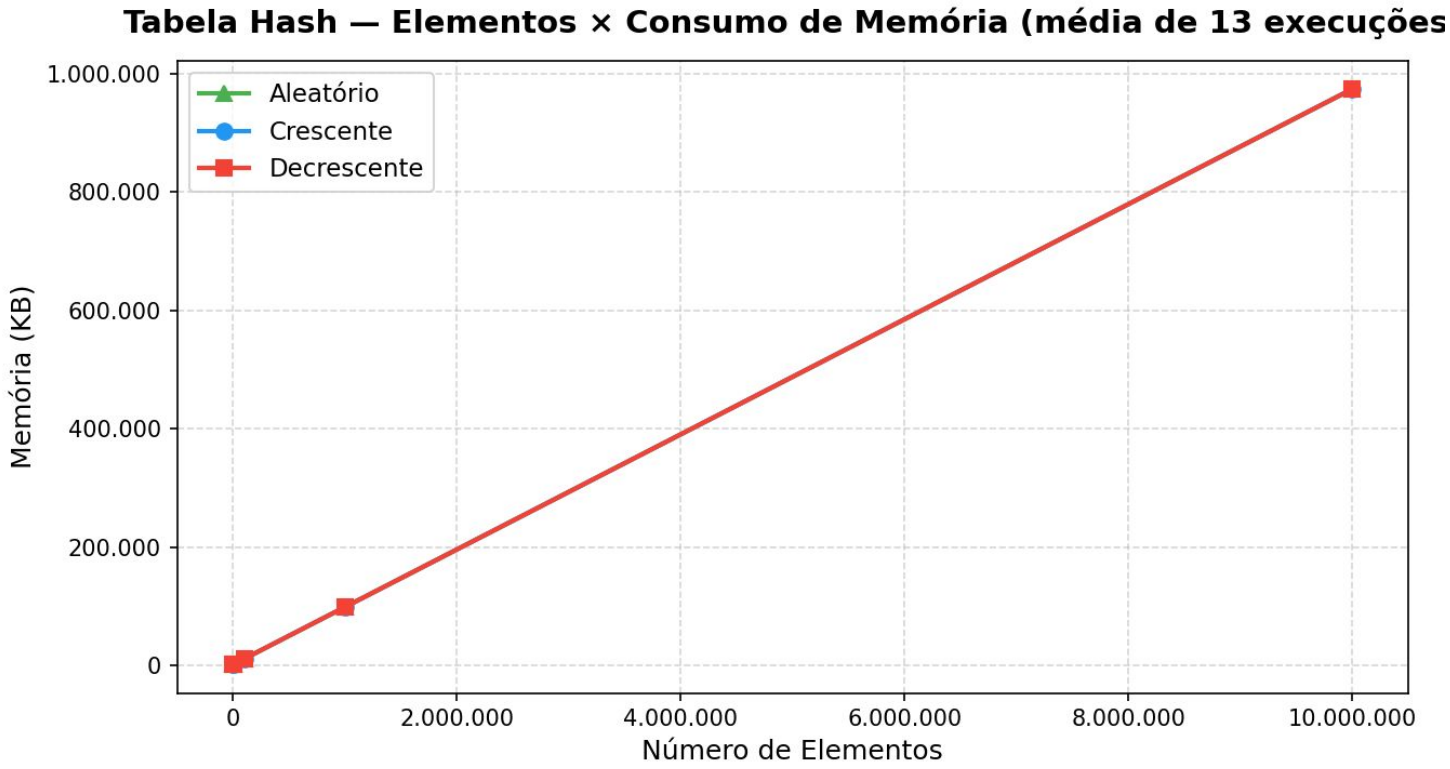


Tabela Hash — Elementos × Consumo de Memória (média de 13 execuções
Aleatório
Crescente
Decrescente
Memória (KB)
1.000.000
800.000
600.000
400.000
200.000
0
0
2.000.000
4.000.000
6.000.000
8.000.000
10.000.000
Número de Elementos

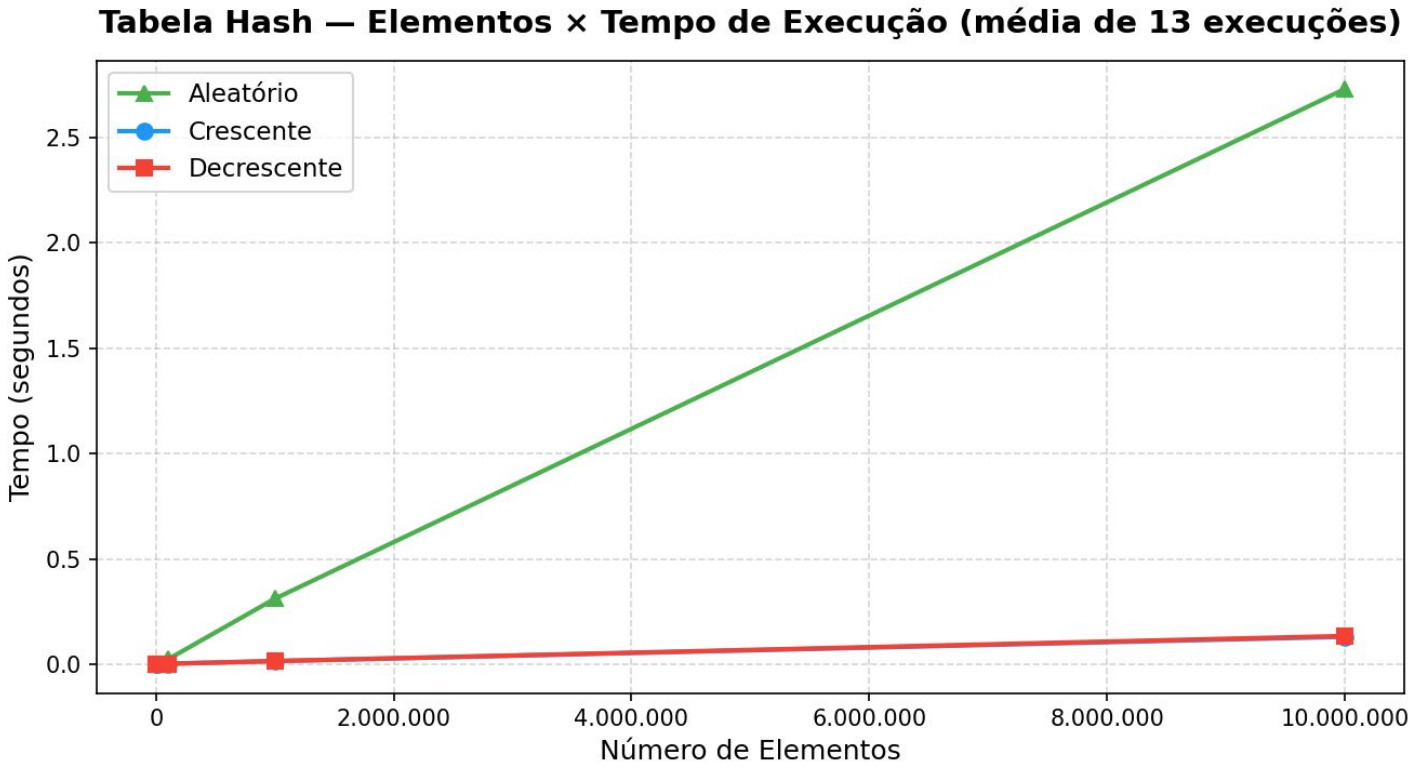
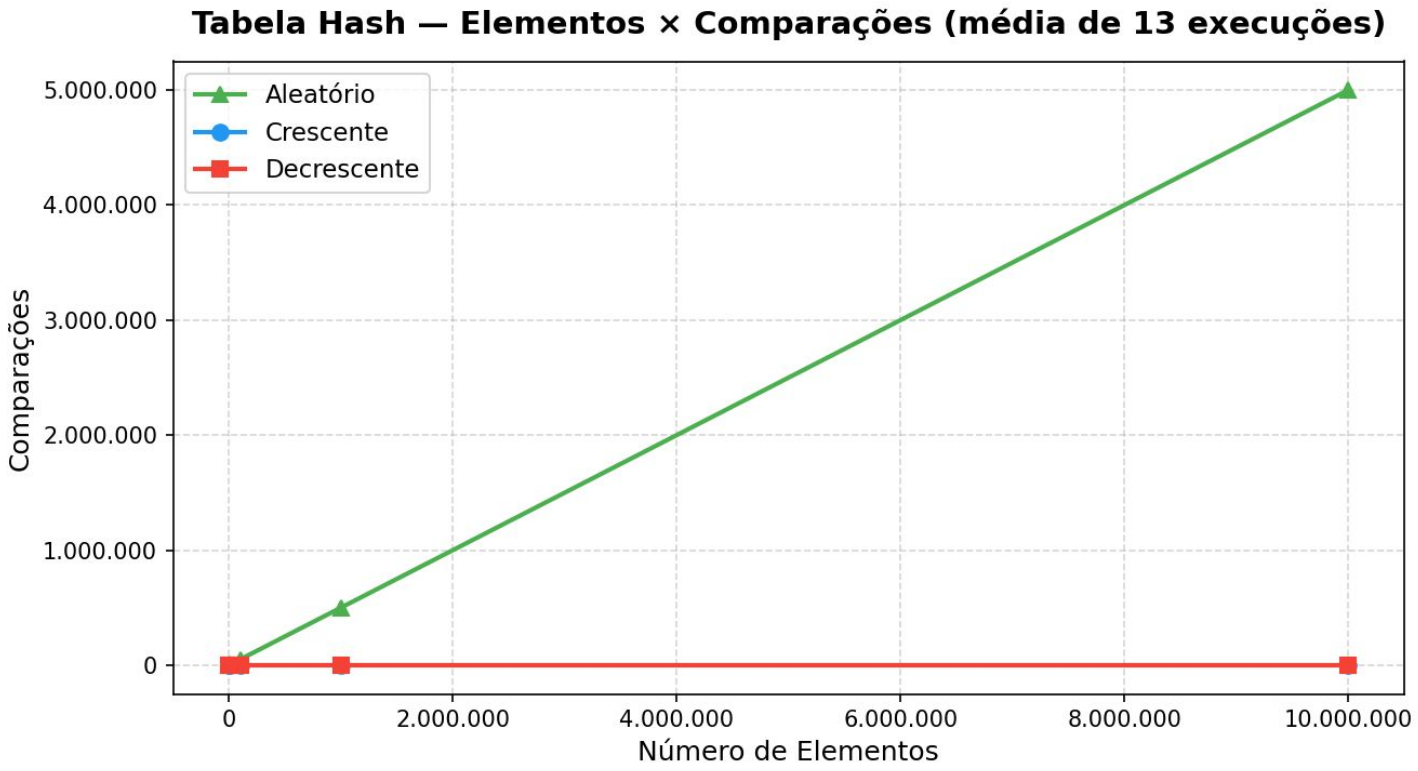


Tabela Hash — Elementos × Tempo de Execução (média de 13 execuções)
Aleatório
Crescente
Decrescente
Tempo (segundos)
2.5
2.0
1.5
1.0
0.5
0.0
0
2.000.000
4.000.000
6.000.000
8.000.000
10.000.000
Número de Elementos



Todos os testes foram realizados em uma máquina virtual linux Mint (Debian) 64-bit com 8192 MB Ram e 8 núcleos de 2.10 GHz